

Coding Style

Contents

- 1 [Formatting & Style](#)
 - 1.1 [Spacing](#)
 - 1.2 [Line indentation, line breaks, and line width](#)
 - 1.3 [Placement of Braces](#)
 - 1.4 ["Yoda conditions" i.e. if \(true == GetState\(\)\)](#)
 - 1.5 [Naming](#)
 - 1.6 [Header vs Cpp](#)
 - 1.7 [#includes](#)
 - 1.8 [Access specifiers](#)
 - 1.9 [TypeDefs](#)
 - 1.10 [Unused function parameters](#)
 - 1.11 [Constness](#)
 - 1.12 [Explicit](#)
- 2 [Clang Format](#)
- 3 [Log message language advice](#)
- 4 [Updated clang-format file](#)

For non-style related guidelines, see [Chip Development Best Practices](#).

Formatting & Style

Spacing

- Avoid TABs.
- Indent using 4 spaces.
- Avoid trailing white space.

Line indentation, line breaks, and line width

- Keep lines below ~160 chars (basis for 160 is that side by side comparison of github diff on DRW standard monitor displays 160 chars)
- If lines become long (generally function definitions, constructor definitions, and function calls), avoid intentional alignment of arguments and parameters. This inevitably creates noise when names change.

```
// Avoid indenting according to name lengths as below
Foo::Foo(int longLongName1, int longLongName2,
        int longLongName3)
{}
```

Instead, pick from a couple of sensible styles (and be consistent with a single one):

```

Foo::Foo(int longLongName1, int longLongName2, int longLongName3)
{}

Foo::Foo(int longLongName1, int longLongName2, int longLongName3,
        int longLongName4)
{}

Foo::Foo(int longLongName1, int longLongName2, int longLongName3)
    : _longLongName1(longLongName1)
    , _longLongName2(longLongName2)
    , _longLongName3(ReallyLongFunctionName(longLongName3))
{}

Foo::Foo(int longLongName1, int longLongName2, int longLongName3,
        int longLongName4)
    : _longLongName1(longLongName1)
    , _longLongName2(longLongName2)
    , _longLongName3(ReallyLongFunctionName(longLongName3))
{}

Foo::Foo(
    int longLongName1, int longLongName2, int longLongName3,
    int longLongName4)
    : _longLongName1(longLongName1)
    , _longLongName2(longLongName2)
    , _longLongName3(ReallyLongFunctionName(longLongName3))
{}

```

- Avoid putting control statements on the same line as the if/else/for/while keyword, and generally avoid putting multiple statements on a single line. This makes debugging more difficult.

```

// Avoid
if(x) return;

// Avoid
while (true) { DoSomething(); DoSomethingElse(); }

```

Placement of Braces

- Place opening curly braces of control flow statements (e.g. if, for, while, etc.) on a new line.
- Place opening curly brace of functions on a new line.
- Lambdas may have their curly brace placed on the same line
- Place opening curly brace of both classes and structs on a new line.
- Place opening curly braces of namespaces on the same line.

Brace Placement

```
namespace SomeNameSpace {

int SomeFunction(int x)
{
    if (x == 0)
    {
        int a = 3 * x - 1;
        return x / a;
    }
    else
    {
        while (x != 0)
        {
            ++x;
            if (x > 0)
                x *= -1;
        }
        return x;
    }
}

auto foo = []() {
    LOG_NOTICE("Lambdas in chip can have a curly brace on the same line");
};

struct SomeClass
{
    int x;
};

} // namespace SomeNameSpace
```

"Yoda conditions" i.e. `if (true == GetState())`

- Please don't.

Naming

- Classes, functions, and namespaces should use capital `PascalCase`.
- Variables and class members use lower `camelCase`.
- Private class members should be prefixed with an `_underscore`.

Naming Convention

```
namespace SomeNameSpace {

class MyClass
{
public:
    int Get() const;
    bool FindValue(int someValue);

public:
    int otherData;

private:
    int _data;
};

} // namespace SomeNameSpace
```

Header vs Cpp

- If a class is a template, and the class is widely read and used, put the definitions at the bottom of the file so that the api is more readable:

```
template <typename T>
class Foo
{
public:
    Foo(A a, B b, C c);

    void DoSomething();

private:
    A _a;
};

template <typename T>
Foo<T>::Foo(A a, B b, C c)
    : _a{a}
{}

template <typename T>
void Foo<T>::DoSomething()
{}

```

- If a class is not a template, prefer to put definitions in .cpp files rather than in the header. (Structs do not need to follow this guideline, but if a struct has many methods it may be better as a class.)
- Exception: if method / function definitions fit neatly on a single line (and do not require the addition of any new headers) - define them in the declaration.

```
// Header
class Foo
{
public:
    Foo(A a, B b, C c);

    void Setter(A a) { _a = a; } // also implicitly requests inline

private:
    A _a;
};

```

- Exception: if a method / function needs to be inlined for performance reasons on a hot path, then it should be defined in the header / declaration.

```
// Header
class Foo
{
public:
    Foo(A a, B b, C c);

    void DoSomethingVeryLatencySensitive()
    {
        // ...
    }

private:
    A _a;
};

```

#includes

- Move #includes to the .cpp file if possible. This helps avoid unnecessary dependencies.
- Prefer forward declarations over additional #includes where possible.

- Use the expanded path to include files even when they are in the same directory as the including file.
- Include headers in the following order:

```
// 1) Paired header
#include "MyProject/MyClass.h"

// 2) Current library headers
#include "MyProject/MyOtherClass.h"
#include "MyProject/MyOtherOtherClass.h"

// 3) Other internal headers in approximate dependency (i.e. topological sort) order
#include "MyProjectDependency/Dependency.h"
#include "Core/Util/Timestamp.h"

// 4) External libraries
#include <boost/intrusive/unordered_set.hpp>
#include <sparsehash/dense_hash_map>

// 5) Standard C++ and C headers
#include <stdint>
#include <vector>

// 6) System headers
#include <semaphore.h>
```

Access specifiers

- Place all public declarations at the top of a class to make its functionality clear.
- When declaring a friend class, put it in its own private or protected section along with the functions it needs access to in the class. This is a form of light documentation.

TypeDefs

- Always prefer using over typedef.
- Avoid aliasing pointer types unless there is a specific need for it (e.g. iterators):

```
struct Foo
{
    using T = Bar*; // Avoid
    using TPtr = Bar*; // Okay if required
    using iterator = Bar*; // Okay if required
};
```

Unused function parameters

- If a parameter is required to be in a function signature (for instance, in a callback), but is unused, do not comment out or omit the parameter name.

```
// Okay
int Foo1(int unused, int used)
{
    return used + 1;
}

// Avoid
int Foo2(int /*unused*/, int used)
{
    return used + 2;
}

// Avoid
int Foo3(int, int used)
{
    return used + 3;
}
```

Constness

- Prefer west-const (const int) over east-const (int const).

Explicit

- Unary constructors should be marked explicit unless they are specifically intended to be used in implicit conversions.

Clang Format

Here is a .clang-format file that matches the Chip style guide.

Disclaimer: The clang-format file might be partially outdated, so that currently it only matches the Chip style 'mostly'. Examples that had objections raised included one member initialization per line.

Copy this file into a readable parent directory of all your git repositories. Specifying ``-style=file`` makes clang-format search from ``$PWD`` to ``/`` for ``.clang-format``. Note that it is currently not possible to specify paths pointing to a custom location of ``.clang-format``.

.clang-format

```
---
Language:      Cpp
# BasedOnStyle: Mozilla
AccessModifierOffset: -4
AlignAfterOpenBracket: DontAlign
AlignConsecutiveAssignments: false
AlignConsecutiveDeclarations: false
AlignEscapedNewlines: Right
AlignOperands: true
AlignTrailingComments: true
AllowAllParametersOfDeclarationOnNextLine: false
AllowShortBlocksOnASingleLine: true
AllowShortCaseLabelsOnASingleLine: false
AllowShortFunctionsOnASingleLine: Inline
AllowShortIfStatementsOnASingleLine: false
AllowShortLoopsOnASingleLine: false
AlwaysBreakAfterDefinitionReturnType: None
AlwaysBreakAfterReturnType: None
AlwaysBreakBeforeMultilineStrings: false
AlwaysBreakTemplateDeclarations: Yes
BinPackArguments: true
BinPackParameters: true
BraceWrapping:
  AfterClass:      true
  AfterControlStatement: true
  AfterEnum:       true
  AfterExternBlock: true
  AfterFunction:   true
  AfterNamespace:  false
  AfterObjCDeclaration: false
  AfterStruct:     true
  AfterUnion:      true
  AfterExternBlock: true
  BeforeCatch:     true
  BeforeElse:      true
  IndentBraces:    false
  SplitEmptyFunction: false
  SplitEmptyRecord: false
  SplitEmptyNamespace: false
BreakBeforeBinaryOperators: None
BreakBeforeBraces: Custom
BreakBeforeInheritanceComma: false
BreakBeforeTernaryOperators: true
BreakConstructorInitializersBeforeComma: false
BreakConstructorInitializers: BeforeComma
BreakAfterJavaFieldAnnotations: false
BreakStringLiterals: false
```

```

ColumnLimit:      160
CommentPragmas:   '^ IWYU pragma:'
CompactNamespaces: true
ConstructorInitializerAllOnOneLineOrOnePerLine: true
ConstructorInitializerIndentWidth: 4
ContinuationIndentWidth: 4
Cpp11BracedListStyle: false
DerivePointerAlignment: false
DisableFormat:    false
ExperimentalAutoDetectBinPacking: false
FixNamespaceComments: false
ForEachMacros:
  - foreach
  - Q_FOREACH
  - BOOST_FOREACH
IncludeBlocks:    Preserve
IncludeCategories:
  - Regex:         '^"(llvm|llvm-c|clang|clang-c)/'
    Priority:      2
  - Regex:         '^(<|"(gtest|gmock|isl|json)/)'
    Priority:      3
  - Regex:         '\. *'
    Priority:      1
IncludeIsMainRegex: '(_tests?)?$'
IndentCaseLabels: false
IndentPPDirectives: None
IndentWidth:      4
IndentWrappedFunctionNames: false
JavaScriptQuotes: Leave
JavaScriptWrapImports: true
KeepEmptyLinesAtTheStartOfBlocks: true
MacroBlockBegin:  ''
MacroBlockEnd:    ''
MaxEmptyLinesToKeep: 1
NamespaceIndentation: None
PenaltyBreakAssignment: 2
PenaltyBreakBeforeFirstCallParameter: 1000000
PenaltyBreakComment: 300
PenaltyBreakFirstLessLess: 120
PenaltyBreakString: 1000000
PenaltyBreakTemplateDeclaration: 10
PenaltyExcessCharacter: 1000000
PenaltyReturnTypeOnItsOwnLine: 200
PointerAlignment: Left
ReflowComments: false
SortIncludes: false
SortUsingDeclarations: false
SpaceAfterCStyleCast: false
SpaceAfterTemplateKeyword: true
SpaceBeforeAssignmentOperators: true
SpaceBeforeCpp11BracedList: false
SpaceBeforeCtorInitializerColon: true
SpaceBeforeInheritanceColon: true
SpaceBeforeParens: ControlStatements
SpaceBeforeRangeBasedForLoopColon: true
SpaceInEmptyParentheses: false
SpacesBeforeTrailingComments: 1
SpacesInAngles: false
SpacesInContainerLiterals: true
SpacesInCStyleCastParentheses: false
SpacesInParentheses: false
SpacesInSquareBrackets: false
Standard:      Cpp11
TabWidth:      4
UseTab:        Never
...

```

You can use it with clang-format version 7:

```
sudo apt install -y clang-format-7

# NOTE: probably should use alternatives system instead
sudo ln -s /usr/bin/clang-format-7 /usr/local/bin/clang-format
sudo ln -s /usr/bin/clang-format-diff-7 /usr/local/bin/clang-format-diff
```

For vim integration, I'm using

```
" clang-format support
Plug 'rhysd/vim-clang-format'

autocmd FileType c,cpp,objc noremap <buffer><Leader>cf :<C-u>ClangFormat<CR>
autocmd FileType c,cpp,objc vnoremap <buffer><Leader>cf :ClangFormat<CR>
```

There are plugins available for sublime and emacs at least; one for eclipse probably exists. This should **only** be run on lines that are being changed; otherwise make a separate commit/PR for whitespace changes, but remember you obfuscate the git blame history doing that.

Log message language advice

JZ: I always try to imagine I'm an App Support person with like 4 months of experience, and what I would feel when I see that log.

The language used for the message should be appropriate to the severity of the log. In case of a non-fatal error, succinctly describe the consequences.

Updated clang-format file

Updated file:

[.clang-format](#)

Worked with

```
$ clang-format --version
LLVM (http://llvm.org/):
  LLVM version 3.4.2
  Optimized build.
  Built Nov  1 2018 (15:06:24).
  Default target: x86_64-redhat-linux-gnu
  Host CPU: x86-64
```